Lab 2: Race Conditions
Geavanceerde Computerarchitectuur

Sien Peerlinck & Thibaut Dalez

March 1, 2024

1 Introduction

In this lab session race conditions were introduced. This topic was explored by implementing a program that finds the maximum value in an array in three different ways: 1) by using a nested for loop on the CPU, 2) by a GPU kernel employing atomic operation and 3) by a GPU kernel using reduction.

In this report all three implementations are explained, the entire code can be found in appendix A.

2 Code

The code begins at line 39 by creating an input array of size N and filling it with random integers. Then this array is put in the GPU's memory, as explained in the report on lab session 1. Afterwards the three different implementations are executed. For assessing timing related questions, we timed the execution of each implementation. We did this with the chrono library by measuring the time right before end right after the execution and then calculating the elapsed time. Timing results and the maximum found, as well as the input array are printed at the end for evaluation. The output for a simple array of size 8 can be found in figure 1

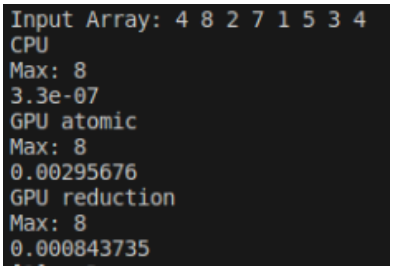


Figure 1: Output for an input array of size 8.

2.1 CPU

The implementation by a nested for-loop on the CPU starts on line 57. One loop iterates over the array and the other (nested) loop compares each element with the elements that come after it. The if function makes sure that only elements bigger then the maximum are put into maxcpu (otherwise if element i is bigger then j it is put into max, but this way the "true" max was replaced in some cases).

2.2 Atomic operation on GPU

The atomic operation makes use of the atomicMax function. This function takes an addresspointer to a value and an integer as arguments and simply replaces the value at the given address with the maximum of both values. The for-loop iterates over the elements of the input array, this allows each thread to handle a portion of the array independently in parallel.

2.3 Reduction on GPU

The reduction starts on line 21. Line 27 compares 2 elements and updates the first element with the max. The for loop iterates through the algorithm to halve the calculations in the array each time. The position of the 2

elements that are compared are calculated with s*position and (s+1)*position (example: 0 and 1, next iteration 0 and 2, ...). The if condition (line 26) checks if the thread can still participate in the calculation (because each time x/2 threads will participate).

3 Conclusion

The cpu speed increases with the size of the array, with the amount of calculations it needs to do. It does so linearly (the x-axis is logarithmic). The atomic max operation on the GPU stays the same for each size of the array, this because it can operate everything in parallel and is not bound by the size of the array (with certain limitations). Except when the array has a size of 96 then the time needed will be 100x times higher. The reduction implementation behaves similar to the atomicMax implementation.

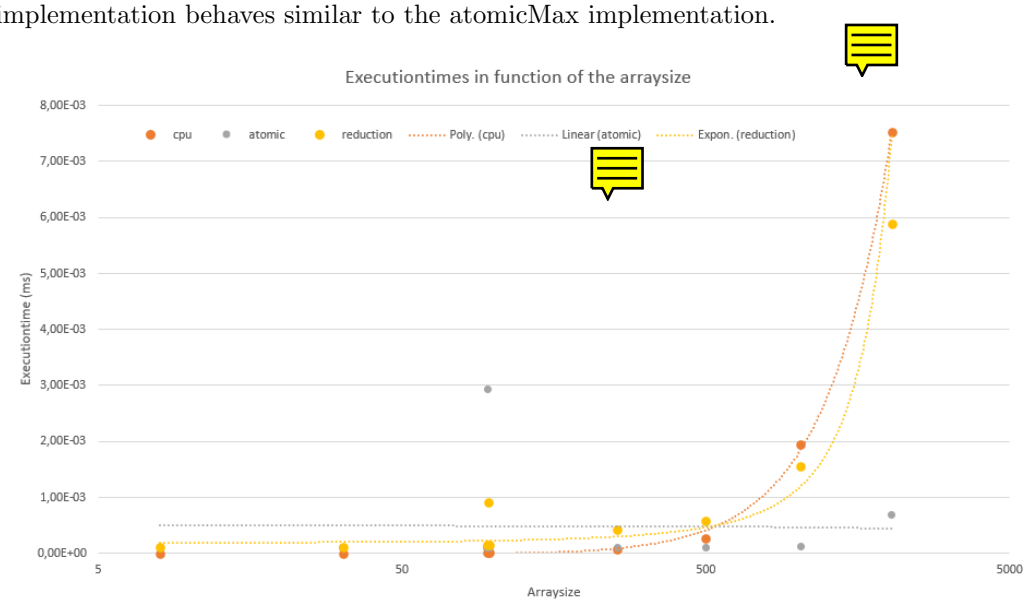


Figure 2: Execution times in function of the arraysize for all three implementations

A Program

```
#include <iostream>
#include <cuda_runtime.h>
#include <chrono>

#define N 96 // Size of the array


__global__ void GPUatomicmax(int *input, int *max2, int size){
    int tid = blockIdx.x * blockDim.x + threadIdx.x;  // total index = number of the block we're in now x number of threads + number of the thread in the current block
    int stride = blockDim.x*gridDim.x;

    int max = 0;

    for(int i = tid; i<size; i+=stride){
        atomicMax(&max,  input[i]);
    }

    atomicMax(max2, max);
}

__global__ void GPUreductionmax(int *input, int *max3, int size){
    unsigned int tid = threadIdx.x;
    __syncthreads();

    for (int s = 1; s < size/2 ; s++) {
        if (tid < s) {
            input[s*tid] = max(input[s*tid], input[s*(tid + 1)]);
        }
        __syncthreads();
    }
     *max3 = input[0];
     //relatie tussen de verschillende -> bij de eerste is *2 en +1, volgende is *4 en +2 , volgende ...

}

// CPU code
int main() {
    // Creation of the input array of size N (defined on line 5) filled with random integers
    int inputArray[N];
    for (int i=0; i<N; i++){
        inputArray[i] = rand();
    }

    // Allocating a memoryplace for the input array, putting it there and moving it to the GPU memory
    int *d_inputArray;
    cudaMalloc(&d_inputArray, N * sizeof(int));
    cudaMemcpy(d_inputArray, inputArray, N * sizeof(int), cudaMemcpyHostToDevice); // cpu memory to GPU memory

    // Define grid and block size
    int blockSize = 256;  // aantal threads in block
    int gridSize = (N + blockSize - 1) / blockSize;

    // Nested for-loop on CPU
    int maxcpu = 0;
    const auto start = std::chrono::steady_clock::now();

    for ( int i = 0; i<N; i++) {
        for(int j=i+1; j<N; j++) {
            if(inputArray[i]>inputArray[j] & inputArray[i]>maxcpu){
                maxcpu = inputArray[i];
            }
        }
    }

    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed_seconds_cpu{end - start};

    //GPU atomic
    int maxgpu1 = 0;
    int *d_maxgpu1;
    cudaMalloc(&d_maxgpu1, sizeof(int));
```

```
    const auto startgpu1 = std::chrono::high_resolution_clock::now();

    GPUatomicmax<<<gridSize, blockSize>>>(d_inputArray, d_maxgpu1, N);  // geef pointers naar
    arrays mee
    cudaMemcpy(&maxgpu1, d_maxgpu1, sizeof(int), cudaMemcpyDeviceToHost); // cpu memory to GPU
     memory

    cudaFree(d_maxgpu1);

    const auto endgpu1 = std::chrono::high_resolution_clock::now();
    const std::chrono::duration<double> elapsed_seconds_gpu{endgpu1 - startgpu1};

    //GPU reduction
    int maxgpu2 = 0;
    int *d_maxgpu2;
    cudaMalloc(&d_maxgpu2, sizeof(int));
    const auto startgpu2 = std::chrono::high_resolution_clock::now();

    GPUreductionmax<<<gridSize, blockSize>>>(d_inputArray, d_maxgpu2, N);

    cudaDeviceSynchronize();

    cudaMemcpy(&maxgpu2, d_maxgpu2, sizeof(int), cudaMemcpyDeviceToHost);

    cudaFree(d_inputArray);
    cudaFree(d_maxgpu2);

    const auto endgpu2 = std::chrono::high_resolution_clock::now();
    const std::chrono::duration<double> elapsed_seconds_gpu2{endgpu2 - startgpu2};

    // Print the result (standaard cpp stuff)
    std::cout << "Input Array: ";
    for (int i = 0; i < N; i++) {
        std::cout << inputArray[i] << " ";
    }
    std::cout << std::endl;
    std::cout << "CPU" << std::endl;
    std::cout << "Max: " << maxcpu << std::endl;
    std::cout << elapsed_seconds_cpu.count() << "\n";

    std::cout << "GPU atomic" << std::endl;
    std::cout << "Max: " << maxgpu1 << std::endl;
    std::cout << elapsed_seconds_gpu.count() << "\n";

    std::cout << "GPU reduction" << std::endl;
    std::cout << "Max: " << maxgpu2 << std::endl;

    std::cout << elapsed_seconds_gpu2.count() << "\n";

    return 0;
}
```

Listing 1: Maximum detection